Koncepcja pracy inżynierskiej

Koncepcja pracy inżynierskiej

Temat: Dynamiczny i semantyczny routing zapytań na podstawie złożoności zadania

Autor: Philip Melkowski

---

1. Cel pracy i rezultat

Cel

Celem pracy jest zaprojektowanie i implementacja systemu dynamicznego routingu zapytań kierowanych do modeli językowych (LLM), który automatycznie klasyfikuje każde zapytanie pod względem złożoności oraz kategorii tematycznej i kieruje je do modelu o odpowiedniej mocy obliczeniowej.

Kontekst problemu

Współczesne frameworki do obsługi modeli językowych — zarówno chmurowe (OpenAI, Anthropic), jak i lokalne (Ollama, vLLM, DeepFellow) — udostępniają wiele modeli o różnej wielkości i wydajności. W praktyce administratorzy stają przed wyborem: albo kierują wszystkie zapytania do największego, najdroższego modelu (co gwarantuje jakość, ale marnuje zasoby GPU i zwiększa latencję), albo do małego modelu (co jest szybkie, ale obniża jakość dla trudnych zapytań). Żadne z tych podejść nie jest optymalne.

Inteligentny router rozwiązuje ten problem: analizuje zapytanie **przed** wywołaniem modelu i kieruje je do modelu o odpowiedniej mocy. Proste pytania (np. „What is a hash table?“) trafiają do lekkiego modelu z latencją rzędu milisekund; złożone zapytania (np. „Implement a concurrent hash map with lock striping and explain the trade-offs”) trafiają do modelu ciężkiego, który potrafi dostarczyć odpowiedź wysokiej jakości.

System powstaje jako natywny komponent frameworka **DeepFellow Infra** — open-source’owego frameworka do orkiestracji modeli AI on-premises, udostępniającego OpenAI-compatible API. Router jest zintegrowany jako nowy typ serwisu (`RouterService`) wewnątrz frameworka, co daje mu bezpośredni dostęp do rejestru modeli bez dodatkowych skoków sieciowych. Użytkownik instaluje serwis routera tak samo jak każdy inny serwis w DeepFellow, konfiguruje dostępne modele i progi przez YAML, a router staje się dostępny jako model pod adresem `/v1/chat/completions`.

Kontekst w literaturze

Problem routingu zapytań LLM jest adresowany przez dwa główne podejścia w literaturze:

RouteLLM (Ong et al., 2024) — system uczony na danych preferencji z Chatbot Arena (80 tys. głosowań). Dla każdego zapytania oblicza prawdopodobieństwo P(win_strong | q), gdzie win_strong oznacza że mocny model da lepszą odpowiedź. Autorzy badają cztery metody estymacji tego prawdopodobieństwa:

- *Similarity-Weighted (SW) Ranking* — porównuje nowe zapytanie z 80 tys. historycznych pytań przez cosine similarity embeddingów; ważona suma głosów daje wynik. Nie wymaga trenowania, ale jest wolna operacyjnie (~39 USD/milion zapytań).

- *Matrix Factorization* — uczy się ukrytych reprezentacji modeli i zapytań (inspiracja systemami rekomendacyjnymi); szybka w inferencji (~155 req/s), wymaga GPU 8 GB.
- *BERT Classifier* — fine-tuning BERT_base na danych preferencji, klasyczna klasyfikacja przez token [CLS] i warstwę liniową; ~70 req/s.
- *Causal LLM Classifier* — fine-tuning Llama 3 8B odpowiadającej jednym tokenem "strong"/"weak"; najdokładniejsza ale najwolniejsza (~42 req/s, 8× GPU A100).

Wszystkie metody RouteLLM wymagają fine-tuningu routera na danych preferencji i obsługują wyłącznie routing binarny (dwa modele, brak obsługi zapytań wieloetapowych).

FrugalGPT (Chen, Zaharia, Zou, 2023) — proponuje trzy strategie redukcji kosztów: skracanie promptów (*Prompt Adaptation*), zastępowanie drogiego modelu tańszym (*LLM Approximation*) oraz kaskadę modeli (*LLM Cascade*). W kaskadzie FrugalGPT zapytanie trafia najpierw do najtańszego modelu; jeśli *Scoring Function* (fine-tunowany DistilBERT) oceni odpowiedź jako niewystarczającą, zapytanie przechodzi do droższego modelu. Kluczowa różnica: FrugalGPT ocenia **odpowiedź** (po wywołaniu modelu), co jest dokładniejsze, ale wolniejsze — każde zapytanie wymaga co najmniej jednego wywołania modelu zanim router podejmie decyzję. Autorzy raportują oszczędności 59–98% przy zachowanej jakości GPT-4 na trzech zbiorach (HEADLINES, OVERRULING, COQA).

Proponowane podejście i różnice względem literatury

Proponowane rozwiązanie łączy zalety obu podejść, eliminując ich główne ograniczenia:

- **Nie wymaga fine-tuningu routera** ani danych preferencji — klasyfikator oparty jest na kaskadzie trzech warstw (heurystyki leksykalne → embeddingi → mały model zero-shot), z których żadna nie wymaga uczenia nadzorowanego.
- **Ocena przed wywołaniem modelu** (jak RouteLLM, w odróżnieniu od FrugalGPT) — router klasyfikuje zapytanie zanim wywoła model docelowy, eliminując dodatkowe wywołania.
- **Dwuwymiarowa klasyfikacja** — oprócz złożoności (SIMPLE/COMPLEX) wykrywana jest kategoria tematyczna (CODING, MATH, REASONING itd.), co umożliwia routing do modeli wyspecjalizowanych per kategoria.
- **Obsługa zapytań wieloetapowych** przez pipeline Map-Reduce — dekompozycja złożonych zapytań na pod-zadania z grafem zależności; żadna z powyższych prac nie adresuje tego problemu.

Rezultat końcowy

Produktem końcowym jest **moduł oprogramowania** zintegrowany z frameworkiem DeepFellow Infra jako natywny serwis (`RouterService`), składający się z trzech komponentów:

1. **Klasyfikator kaskadowy** — trójwarstwowy system oceny złożoności i kategorii zapytania: heurystyki → embeddingi → model zero-shot. Każdy wymiar (złożoność i kategoria) jest rozstrzygany niezależnie — kolejna warstwa kaskady uruchamia się wyłącznie dla wymiarów, które nie zostały ustalone z wystarczającą pewnością.
2. **Router** — logika wyboru modelu docelowego na podstawie wyniku klasyfikatora (złożoność × kategoria), konfigurowana przez macierzowy plik YAML. W zakresie pracy obsługiwane są dwa modele docelowe (Qwen 1.5B i Llama 3 70B); architektura wspiera rozszerzenie o modele wyspecjalizowane per kategoria wyłącznie przez zmianę konfiguracji, bez modyfikacji kodu.
3. **Pipeline Map-Reduce** — dekompozycja złożonych zapytań wieloetapowych na pod-zadania, ich niezależne przetwarzanie i scalenie wyników.

Rezultat jest weryfikowalny: da się go uruchomić jako endpoint HTTP (`/v1/chat/completions?model=router`), zmierzyć dokładność klasyfikacji na zbiorze testowym i porównać metryki jakości odpowiedzi z architekturą statyczną.

Poza zakresem

Praca nie obejmuje: fine-tuningu żadnego z modeli językowych, obsługi konwersacji wieloturowych (system operuje na poziomie pojedynczego zapytania), wdrożenia produkcyjnego w skali horyzontalnej, zapytań w językach innych niż angielski ani ewaluacji modeli wyspecjalizowanych per kategoria (macierz routingu wskazuje na te same dwa modele; specjalizacja jest zaprojektowana jako rozszerzenie).

---

2. Technologie i sposób integracji

Stos technologiczny

Język i runtime — Python 3.13 z menedżerem pakietów `uv`, spójny ze stackiem DeepFellow Infra.

Integracja z frameworkiem — `RouterService` dziedziczący `Base2Service` — natywna klasa serwisu z pełnym dostępem do `EndpointRegistry`. Jedyna modyfikacja kodu DF Infra to wpięcie nowego serwisu w `lifecycle.py`.

Tokenizacja (L1) — `tiktoken` (cl100k_base). Szybka implementacja BPE w Rust (latencja mikrosekund); dobrze obsługuje bloki kodu i URL-e w odróżnieniu od prostego podziału po spacjach. Wystarczy jeden tokenizator, bo token count służy wyłącznie jako przybliżenie długości — nie jest inputem do żadnego modelu, a progi kalibrowane są na tych samych tokenach.

Embeddingi (L2) — `nomic-embed-text` wywoływany in-process przez `endpoint_registry.execute_embeddings()`. Dostępny lokalnie w Ollama, 2048 tokenów kontekstu, trenowany na text retrieval.

Classifier zero-shot (L3) — Qwen 1.5B wywoływany in-process przez `endpoint_registry.execute_chat_completion()`. Mały model (~10–20 ms), lokalny, brak zależności od external API.

Modele docelowe — Qwen 1.5B (model SIMPLE) i Llama 3 70B (model COMPLEX), oba zainstalowane w DeepFellow i wywoływane in-process.

Konfiguracja — PyYAML. Progi, wagi i macierz routingu zmieniane bez recompile.

Dane — `lmsys/chatbot_arena_conversations` (HuggingFace, gated access) jako publiczny dataset z głosowaniami ludzi, zgodny z metodologią RouteLLM. Dane syntetyczne i ewaluacja LLM-as-Judge przez Gemini 2.5 Flash API (free tier, 1500 req/dzień).

Ewaluacja — `rouge-score` (biblioteka) dla zadań ekstrakcji; GPT-as-Judge w skali 1–5 (Gemini 2.5 Flash) dla rozumowania i kodu.

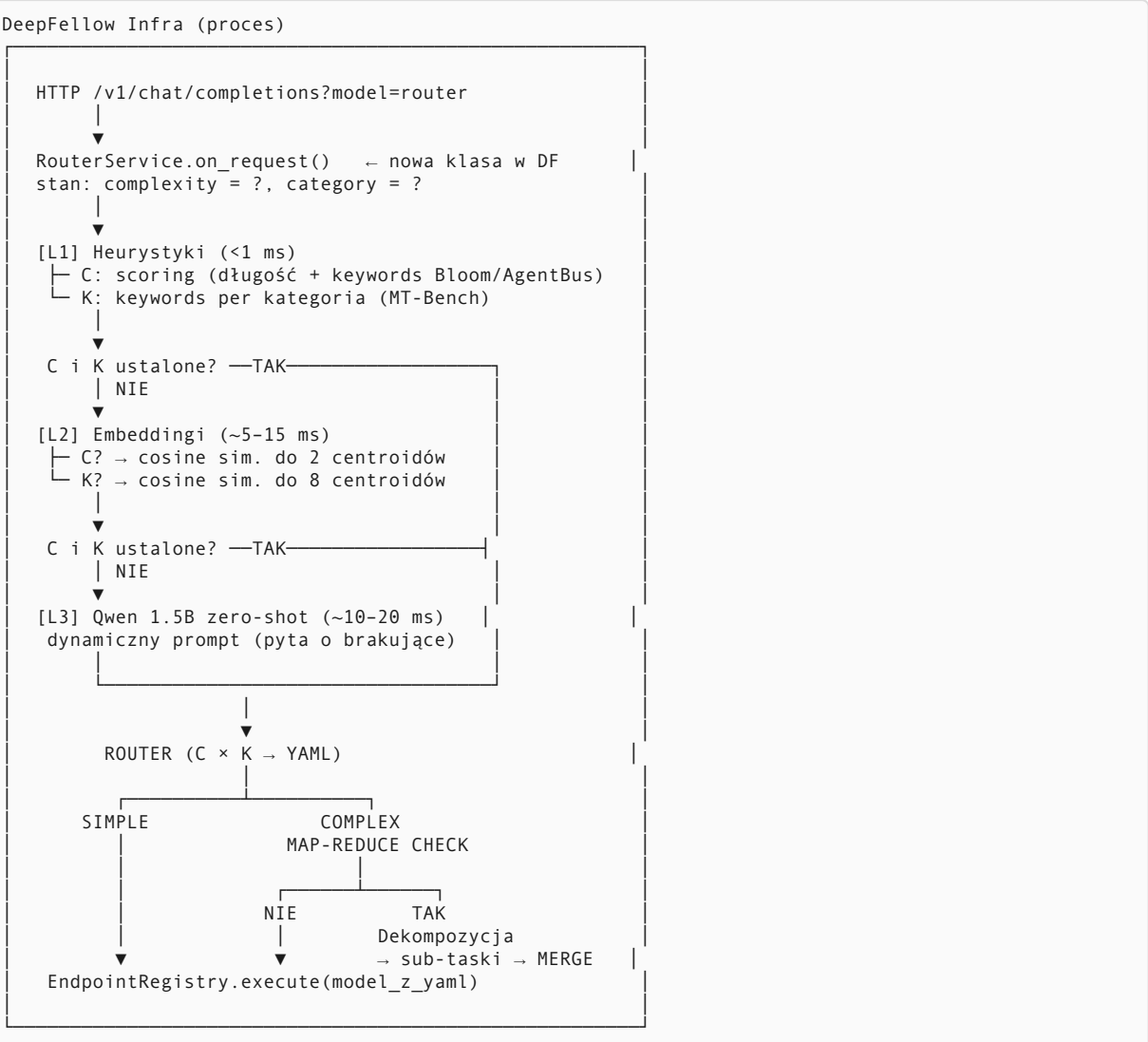
Gdzie kończy się kod własny, a zaczyna cudzy

Praca obejmuje implementację trzech modułów: klasyfikatora kaskadowego, routera i pipeline'u Map-Reduce. Całość jest opakowana w jedną klasę `RouterService`, która zostaje dodana do DeepFellow Infra jako nowy typ serwisu — jest to jedyna modyfikacja kodu frameworka (wpięcie w `lifecycle.py`).

Modele językowe (Qwen 1.5B, Llama 3 70B) i model embeddingów (nomic-embed-text) to istniejące komponenty frameworka — router ich nie instaluje ani nie konfiguruje, a jedynie wywołuje po nazwie przez wewnętrzny rejestr endpointów. Z perspektywy routera są to czarne pudła z interfejsem OpenAI-compatible.

2a. Wstępna architektura

Architektura opiera się na kaskadzie trzech warstw, z których każda rozstrzyga niezależnie dwa wymiary: **złożoność** (C — SIMPLE/COMPLEX) i **kategorię** (K — CODING, MATH, REASONING, WRITING, ROLEPLAY, EXTRACTION, KNOWLEDGE_STEM, KNOWLEDGE_HUMANITIES lub UNKNOWN). Kolejna warstwa uruchamia się wyłącznie dla wymiarów nieustalonych z wystarczającą pewnością.



Użytkownik przy instalacji serwisu konfiguruje tryb pracy i dostępne modele. Najprostszy wariant to dwa modele (lekki i ciężki) bez rozróżniania kategorii; pełny wariant pozwala przypisać modele per kategoria i poziom złożoności.

Fragment konfiguracji — macierz routingu (`config/router.yaml`):

```yaml
routing:
  CODING:
    SIMPLE:  qwen-1.5b
    COMPLEX: llama-70b
  MATH:
    SIMPLE:  qwen-1.5b
    COMPLEX: llama-70b
  UNKNOWN:
    SIMPLE:  qwen-1.5b
    COMPLEX: llama-70b
```

W zakresie pracy wszystkie komórki wskazują na te same dwa modele. Podmiana na modele wyspecjalizowane (np. `deepseek-coder` dla CODING) wymaga wyłącznie zmiany YAML — bez modyfikacji kodu routera.

Wszystkie wywołania modeli (L2, L3, SIMPLE, COMPLEX) odbywają się **in-process** przez `EndpointRegistry` — bez dodatkowych skoków HTTP. Punkt integracji z frameworkiem: `RouterService` wpisany w `lifecycle.py` DeepFellow Infra — jedyne miejsce modyfikacji kodu frameworka.

Szczegóły warstw klasyfikatora

Warstwa 1 — Heurystyki. System punktowy (scoring) oparty na trzech sygnałach: długości promptu w tokenach (progi: 50/150/400 tokenów, +1/+2/+3 punkty), obecności słów kluczowych z dwóch zewnętrznych źródeł (Bloom's Taxonomy — poziomy Apply, Analyze, Evaluate, Create; oraz AgentBus — praktyczny system routingu produkcyjnego) i liczbie niezależnych poleceń w zapytaniu. Suma punktów porównywana z progiem THRESHOLD; pewność = znormalizowana odległość sumy od progu. Równolegle, osobny zestaw słów kluczowych per kategoria (ze źródeł MT-Bench) wykrywa kategorię tematyczną; brak pewnego sygnału oznacza UNKNOWN. Wszystkie progi i wagi konfigurowane w YAML.

Kandydaci na słowa kluczowe złożoności zweryfikowani empirycznie na zbiorze walidacyjnym (1841 rekordów) z progami precision ≥ 80% i recall ≥ 1%. Frazy które przeszły weryfikację: `develop`, `analyze`, `step by step`, `design`, `plan`, `recommend`, `solve`, `build`, `implement`, `generate`, `evaluate`. Frazy z niską precision (np. `calculate` 58.6%, `compose` 41.7%) odrzucone — aktywnie wprowadzają błędy.

Warstwa 2 — Embeddingi. Zapytanie wektoryzowane przez `nomic-embed-text` i porównywane z centroidami wyznaczonymi ze zbioru walidacyjnego: 2 centroidy złożoności (SIMPLE/COMPLEX) i 8 centroidów kategorii (per kategoria MT-Bench). Pewność = różnica cosine similarity między dwoma najbliższymi centroidami. Porównanie wykonywane tylko dla nieustalonych wymiarów.

Warstwa 3 — Model zero-shot. Qwen 1.5B odpytywany dynamicznym promptem, zawierającym wyłącznie pytania o nieustalone wymiary (np. "Classify complexity: SIMPLE or COMPLEX" jeśli kategoria już znana). Pewność = softmax zwycięskiego tokenu. Koniec kaskady — zawsze zwraca decyzję.

---

3. Wyzwania i niewiadome

Techniczne

Kalibracja progów kaskady. Kluczowym parametrem każdej warstwy jest próg confidence decydujący czy wynik jest wystarczająco pewny by zakończyć klasyfikację danego wymiaru. Z dwoma wymiarami (złożoność + kategoria) przestrzeń kalibracji się powiększa — dla każdej warstwy osobny próg per wymiar. Zbyt niski próg oznacza, że niepewne przypadki nie trafiają do kolejnej warstwy; zbyt wysoki oznacza, że kaskada traci sens bo większość zapytań i tak dochodzi do L3. Kalibracja wymagać będzie przeszukania siatki parametrów (grid search) na zbiorze walidacyjnym.

Qwen 1.5B jako zero-shot classifier. Nie wiadomo z góry, czy model 1.5B parametrów jest wystarczająco dobry do oceny złożoności i kategorii zapytania bez fine-tuningu. Artykuł o interpretowalności mechanistycznej (arXiv:2602.17229) raportuje ~95% skuteczności wewnętrznych aktywacji modelu w klasyfikacji poziomu Bloom's Taxonomy — ale przy użyciu linear probing, nie zero-shot. Może okazać się konieczna iteracja nad promptem systemowym lub użycie większego modelu w L3.

Map-Reduce a latencja. Dekompozycja zapytania przez Qwen 1.5B plus równoległe wywołania routera dla pod-zadań mogą być łącznie droższe czasowo niż bezpośrednie wywołanie Llama 70B. FrugalGPT potwierdza to zjawisko — kaskada jest zawsze wolniejsza od pojedynczego wywołania najdroższego modelu; opłacalność zależy od proporcji zapytań które kończą się na wcześniejszych, tańszych etapach. Próg opłacalności Map-Reduce wymaga empirycznego wyznaczenia.

Integracja z kodem DeepFellow Infra. Dodanie `RouterService` wymaga modyfikacji `lifecycle.py` (wpięcie w startup) i potencjalnie `applicationcontext.py`. To jedyne miejsce styku z kodem frameworka — ryzyko: zmiany w API DF Infra mogą wymagać aktualizacji integracji; konieczna koordynacja z rozwojem frameworka.

Dobór listy słów kluczowych (L1) — złożoność. Lista kandydatów pochodzi z Bloom's Taxonomy i AgentBus. Trzy metodologie budowy finalnej listy (empiryczna, literaturowa, hybrydowa) są otwartą decyzją wpływającą na metodologiczne uzasadnienie w pracy.

Dobór słów kluczowych (L1) — kategoria. Kandydaci z opisów kategorii MT-Bench; weryfikacja empiryczna na zbiorze walidacyjnym analogicznie do złożoności. Ryzyko: kategorie o nakładającej się leksyce (KNOWLEDGE_STEM vs KNOWLEDGE_HUMANITIES, WRITING vs EXTRACTION) mogą nie osiągnąć progu precision — trafią do UNKNOWN i obsłuży je model domyślny. To akceptowalne zachowanie; heurystyki nie muszą wykrywać wszystkich kategorii — nierozstrzygnięte przypadki eskalowane są do warstwy embeddingów lub modelu zero-shot.

Dane

Niezbalansowany dataset. Zbiór finalny zawiera 2632 rekordy: COMPLEX 64% (1687), SIMPLE 36% (945). Wymaga to ważonego F1 i ostrożnej interpretacji accuracy. Rozkład kategorii jest również nierównomierny (od 162 rekordów ROLEPLAY do 339 WRITING) — przy 8 centroidach kategorii mniejsze klasy mogą mieć mniej reprezentatywne centroidy.

Jakość etykiet z Chatbot Arena. Etykiety wynikają z głosowań ludzkich sędziów porównujących modele o różnym Elo, nie z eksplicytnej oceny złożoności. Autorzy RouteLLM sami wskazują na ograniczenia tych danych — rozkład produkcyjny może się różnić od danych Areny, a benchmarki mogą być zbyt łatwe (MT Bench to 80 pytań konwersacyjnych). W pracy ryzyko adresowane jest przez augmentację danymi syntetycznymi generowanymi przez Gemini 2.5 Flash.

Infrastrukturalne

Dostępność GPU dla Llama 3 70B. Testy end-to-end i benchmarki wymagają działającej instancji Llama 70B w DeepFellow. Zależność od dostępności sprzętu może opóźnić ewaluację.

Limit API Gemini. Gemini 2.5 Flash na free tier: 1500 req/dzień. Przy dużym zakresie ewaluacji jakości (GPT-as-Judge) konieczne rozłożenie na kilka dni lub priorytetyzacja próbki.

---

4. Kryteria sukcesu i harmonogram

Kryteria akceptacji

Metryka	Cel	Metoda weryfikacji
F1 (klasa COMPLEX)	≥ 0.85	zbiór testowy (791 rek.)
Błąd COMPLEX → SIMPLE	< 10%	macierz pomyłek
PGR routera	≥ 0.75	porównanie z baselinem
Overhead latencji	< 50 ms (p90)	benchmarki systemowe

PGR (Performance Gap Recovered) — metryka z RouteLLM mierząca ile z różnicy jakości między słabym a mocnym modelem udaje się odzyskać: PGR = (jakość_routera − jakość_słabego) / (jakość_mocnego − jakość_słabego). Losowy routing daje PGR ≈ 0.5. Najlepsze wyniki RouteLLM to APGR = 0.802 na MT Bench (Matrix Factorization z augmentacją) — proponowany system dąży do PGR ≥ 0.75 bez fine-tuningu routera. Dodatkowe metryki z RouteLLM (APGR — pole pod krzywą PGR, CPT(X%) — minimalny procent zapytań do mocnego modelu dla danego PGR) będą raportowane w celu porównania z literaturą.

Jakość odpowiedzi mierzona przez ROUGE (dla zapytań ekstrakcji i tłumaczenia) i GPT-as-Judge w skali 1–5 (Gemini 2.5 Flash, dla zapytań wymagających rozumowania i kodu).

Plan realizacji

Etap	Opis	Status
Stage 1 — Dane	Filtrowanie Chatbot Arena, generowanie danych syntetycznych, budowa datasetu (2632 rek.), split 70/30	zrealizowany
Stage 2 — Klasyfikator	Kaskada: heurystyki (L1), embeddingi (L2) z 10 centroidami, Qwen zero-shot (L3)	wakacje 2026
Stage 3 — Router	Logika wyboru modelu, integracja `RouterService` w DF, konfiguracja YAML	wakacje 2026
Stage 4 — Map-Reduce	Heurystyka rozkładalności, dekompozycja, strategie merge	wakacje 2026
Stage 5 — Ewaluacja	Kalibracja progów, pomiar metryk, benchmarki systemowe	wakacje 2026
Stage 6 — Integracja	Testy end-to-end, dokumentacja, pisanie pracy	wakacje 2026